# Assignment 2

# Digital Signal processing

# FIR filters

Bernd Porr, Georgina Klemencic, Scott Watson.

Form groups of 2 people.

The task of this assignment is to filter an ECG with FIR filters and to detect the R peaks. In contrast to the FFT assignment we write filter code which can be used for realtime processing. This means that the FIR filter needs to be implemented with the help of delay lines and the impulse response is truncated.

## ECG filtering

Record an ECG in the lab. Follow the instructions on moodle how to record it.

1. Plot the ECG in the time- and frequency domain (linear axis) and identify the ECG and the noise in the spectrum. There is certainly baseline wander and 50Hz interference but there might be other interferences. Decide which frequency components need to be removed. Identify these frequencies representing noise using your judgement and knowledge about harmonics. **[20%]**
2. Create a general purpose function which calculates the FIR coefficients by providing the sampling rate and an array of frequency ranges which need to be removed also taking care of edge cases such as DC removal. The design function should use the IFFT to calculate the coefficients and also calculates the number of coefficients. Explain your reasoning of how the number of coefficients are calculated. **[25%]**
3. Create a Python FIR filter class which implements an FIR filter which has a method of the form value dofilter(self,value) where both the value argument and return value are *scalars* and not vectors (!) so that it can be used in a realtime system. The constructor of the class takes the coefficients as its input:

   ```
   class FIRfilter:
   def __init__(self,_coefficients):
   # your code here
   def dofilter(self,v):
   # your code here
   return result
   ```

   Filter your ECG with the above FIR filter class using the coefficients from 2. Simulate realtime processing by feeding the ECG sample by sample

into your FIR filter class. Make sure that the ECG looks intact and that it is not distorted (PQRST intact). Provide appropriate plots. **[25%]**

4. Instead of using bandstop filters to remove noise use an adaptive LMS filter by providing it with a noise reference. Add an adaptive LMS filter command to your FIR filter class and name it: "doFilterAdaptive(self,signal,noise,learningRate)" which returns the cleaned up ECG. As before also this function must receive only scalars (i.e. sample by sample) and return a scalar. Plot and compare the result from the adaptive filter and that from the FIR filter design. **[20%]**

5. ECG heartbeat detection The task is to detect R-peaks in your ECG. Write a matched filter which uses a *real ECG as a template*. This template should come from the ECG of the *other* lab partner. Plot the momentary heartrate (i.e. inverse intervals between R-peaks) against time. Remove any wrong heartrates by employing a heuristic, for example, that the heartrate can't be higher than 200bps, etc. **[20%]**

Every report must be based on the recordings made by this team. Please keep it short but it should make clear what you have done and *why* you have done it. Include the complete Python code as well as plots of the ECGs (timedomain) and their frequency representation (with proper labels). If necessary annotate the plots with InkScape by labelling the ECG peaks and remember to use vector based image formats, for example EPS, SVG, PDF or EMF and not pixel based formats for the report. These figures need to be in the report at the correct place and not attached separately. Also, show zoomed in ECG traces of one heartbeat so that it is possible to identify the different parts of a single heartbeat and that it's possible to check if it's still intact.

Make sure your code is:

- platform-independent
- does not contain absolute paths
- Code that crashes will result in low marks. The same applies to code which does not display any plots.
- No high-level Python signal processing/filtering commands are allowed (including, for example, fftfreq) except for the FFT and IFFT commands. Any use of high-level code will result in zero marks for that part.
- No high level Python functions except of FFT/IFFT and the window functions are allowed. Any use of "lfilter", "firwin", "conv", "correl" and any a-causal processing (i.e. data array in and data array out) commands will result in zero or very low marks. Do not use FFT- or IIR-filtering in this assignment. The purpose of this assignment is to assess your use of the FIR filter, so filtering must be carried out by designing and using an FIR filter. Any use of FFT- or IIR-filtering will result in zero marks for that part.
- The code needs to be efficient. For example, shifting all values of the FIR delay line at every time-step will result in lower marks.
- Inflated/unreadable code generated by an LLM will result in low or zero marks. It is important that a human can understand the code generated. All tasks above can be solved elegantly with very few lines of code. If you use GenAI, then take its raw output and optimise it.

- GenAI tools may be used in this assignment. If you use GenAI, you must include a short acknowledgement in your report stating which tool(s) you used (name, version, publisher), how you used them, and what work you contributed yourself. See the UoG guidance on acknowledging GenAI use.
- We reserve the right to interview you to test if you own the code (i.e. that you can explain it).

As before submit a zip file containing all files and test the zip before submission by unzipping it and then running python from the command-line in a terminal (not spyder). Also check that all plots are generated when running the script from the commandline. See moodle for the exact filename conventions.

Deadline is 9th November, 3pm.